5. Python

▼ Python Tasks

Python Use-Cases

A curated collection of project ideas to sharpen your Python skills beyond the basics.

---

1. Web Scraper with Anti-Bot Bypass

Description: Build a scraper that extracts structured data from dynamic, JavaScript-rendered websites. Handle pagination, rate limiting, retries, and rotating user-agents to avoid detection.

Prerequisites:

- HTTP methods, status codes, headers, and cookies
- HTML/CSS selectors (`BeautifulSoup`, `lxml`)
- `requests` or `httpx` library
- `Selenium` or `Playwright` for JS-rendered pages
- Basic `asyncio` for concurrent requests
- Regular expressions for pattern extraction

Use-Case:

- Scrape e-commerce product listings on a nightly schedule
- Store product data (name, price, SKU) in SQLite/PostgreSQL
- Compare against previous day's data and flag price changes

- Export a daily price-change report as CSV

Expected Output:

```
[2026-02-24 02:00:01] Scraper started — target: electronic
s.example.com
[2026-02-24 02:00:03] Page 1/47 — 24 products extracted
[2026-02-24 02:00:06] Page 2/47 — 24 products extracted
...
[2026-02-24 02:12:44] Page 47/47 — 11 products extracted
[2026-02-24 02:12:45] Total: 1,107 products saved to DB

=== Price Change Report ===
+--------------------------------+-----------+-----------+-
-------+
| Product                        | Old Price | New Price |
Change |
+--------------------------------+-----------+-----------+-
-------+
| Sony WH-1000XM5 Headphones   | $348.00   | $279.99   | -
19.5% |
| Samsung Galaxy S25 Ultra      | $1,299.99 | $1,199.99 |
-7.7% |
| Logitech MX Master 3S        | $99.99    | $109.99   | +
10.0% |
+--------------------------------+-----------+-----------+-
-------+
3 price changes detected. Report saved to reports/2026-02-
24.csv
```

---

2. Real-Time Chat Application with WebSockets

Description: Create a multi-room chat server using WebSockets. Support private messaging, typing indicators, user presence tracking, and message

history persistence.

Prerequisites:

- TCP/IP and WebSocket protocol basics
- `asyncio` and `async/await` syntax
- `websockets` or `FastAPI WebSocket` library
- JSON serialization/deserialization
- SQLite or Redis for message persistence
- Basic HTML/JS for the client UI

Use-Case:

- Users connect via browser and join named chat rooms
- Support direct messages between two users
- Show real-time "typing..." indicators
- Display online/away/offline presence status
- Persist and search message history

Expected Output:

```
=== Server Log ===
[INFO] Chat server started on ws://0.0.0.0:8765
[INFO] User "alice" connected (session: a3f8c1)
[INFO] User "bob" connected (session: d92eb4)
[INFO] alice joined room #general
[INFO] bob joined room #general

=== Client View (Alice) ===
#general | 2 members online
────────────────────────────────
[14:32:01] bob: Hey team, anyone available for a code revi
ew?
[14:32:05] bob is typing...
[14:32:08] alice: Sure! Send me the PR link.
```

```
[14:32:15] bob: https://github.com/org/repo/pull/142
──────────────────────────────
Online: alice, bob | carol (away)

=== Private Message (Bob -> Alice) ===
[DM] bob -> alice: Thanks for the quick review!
[DM] alice -> bob: No problem!
```

---

3. Custom ORM (Object-Relational Mapper)

Description: Design a lightweight ORM from scratch using Python metaclasses and descriptors. Support model definition, field validation, query building, relationships, and lazy loading.

Prerequisites:

- Python metaclasses (`__new__`, `__init_subclass__`)
- Descriptor protocol (`__get__`, `__set__`, `__set_name__`)
- Decorators and class decorators
- SQL syntax (DDL + DML)
- `sqlite3` standard library module
- Method chaining pattern

Use-Case:

- Define database tables as Python classes with typed fields
- Auto-generate `CREATE TABLE` SQL from class definitions
- CRUD operations via `.save()`, `.delete()`, `.filter()`
- Support `ForeignKey` relationships with lazy-loaded access
- Chain queries: `User.filter(age__gte=25).order_by("-name").all()`

Expected Output:

```
# --- Developer Usage ---
class User(Model):
    name = CharField(max_length=100)
    email = CharField(max_length=255, unique=True)
    age = IntegerField(nullable=True)

class Post(Model):
    title = CharField(max_length=200)
    author = ForeignKey(User, related_name="posts")

# --- Runtime Output ---
>>> User.create_table()
SQL: CREATE TABLE IF NOT EXISTS user (
       id INTEGER PRIMARY KEY AUTOINCREMENT,
       name VARCHAR(100) NOT NULL,
       email VARCHAR(255) NOT NULL UNIQUE,
       age INTEGER
     );
Table 'user' created.

>>> alice = User(name="Alice", email="alice@example.com",
age=30)
>>> alice.save()
SQL: INSERT INTO user (name, email, age) VALUES ('Alice',
'alice@example.com', 30);
Record saved: User(id=1)

>>> users = User.filter(age__gte=25).order_by("-name").all
()
SQL: SELECT * FROM user WHERE age >= 25 ORDER BY name DES
C;
[User(id=1, name='Alice', email='alice@example.com', age=3
0)]

>>> alice.posts  # lazy-loaded relationship
```

```
SQL: SELECT * FROM post WHERE author_id = 1;
[Post(id=1, title='Hello World', author_id=1)]
```

---

4. Distributed Task Queue

Description: Implement a producer-consumer task queue that distributes work across multiple worker processes. Include task serialization, retry logic with exponential backoff, dead-letter queues, and result backends.

Prerequisites:

- `multiprocessing` and `threading` modules
- `pickle` and `json` serialization
- Redis (via `redis-py`) as a message broker
- Socket programming basics
- Exponential backoff algorithm
- Producer-consumer and pub/sub patterns

Use-Case:

- Producer enqueues callable tasks with arguments
- Multiple worker processes poll the queue and execute tasks
- Failed tasks retry up to N times with increasing delays
- Permanently failed tasks move to a dead-letter queue
- Results stored in a backend (Redis/SQLite) for later retrieval
- Dashboard view showing task status, retries, and duration

Expected Output:

```
=== Broker ===
[BROKER] Listening on redis://localhost:6379/0
[BROKER] Queue "default" — 0 pending, 3 workers connected
```

```
=== Producer ===
>>> queue.enqueue(generate_thumbnail, image_id=4521, size=
(256,256))
Task queued: <Task id=a8f3c1 func=generate_thumbnail statu
s=PENDING>

>>> queue.enqueue(send_email, to="bob@co.com", template="w
elcome")
Task queued: <Task id=b7d4e2 func=send_email status=PENDIN
G>

=== Worker 1 ===
[WORKER-1] Picked up task a8f3c1 (generate_thumbnail)
[WORKER-1] Task a8f3c1 completed in 1.34s — result: /thumb
s/4521_256x256.jpg

=== Worker 2 ===
[WORKER-2] Picked up task b7d4e2 (send_email)
[WORKER-2] Task b7d4e2 FAILED (SMTPConnectionError) — retr
y 1/3 in 2s
[WORKER-2] Task b7d4e2 FAILED (SMTPConnectionError) — retr
y 2/3 in 4s
[WORKER-2] Task b7d4e2 completed in 6.82s — result: email_
sent

=== Dashboard ===
+----------+--------+-----------+--------+-------------+
| Task ID  | Func   | Status    | Retries| Duration    |
+----------+--------+-----------+--------+-------------+
| a8f3c1   | thumb  | SUCCESS   | 0      | 1.34s       |
| b7d4e2   | email  | SUCCESS   | 2      | 6.82s       |
| c9e5f3   | report | DEAD_LETTER| 3     | —           |
+----------+--------+-----------+--------+-------------+
```

5. Machine Learning Pipeline with Feature Engineering

Description: Build an end-to-end ML pipeline that ingests raw data, handles missing values, engineers features, trains multiple models, tunes hyperparameters, and evaluates with cross-validation.

Prerequisites:

- `pandas` for data manipulation
- `numpy` for numerical operations
- `scikit-learn` (pipelines, transformers, model selection)
- Feature engineering techniques (binning, one-hot encoding, scaling)
- Cross-validation and hyperparameter tuning (`GridSearchCV`)
- Evaluation metrics (accuracy, precision, recall, F1, ROC-AUC)

Use-Case:

- Load raw customer data (usage logs, billing, support tickets)
- Impute missing values and remove outliers
- Engineer derived features (e.g., `avg_monthly_spend`, `days_since_last_login`)
- Train and compare Logistic Regression, Random Forest, XGBoost, SVM
- Select best model based on F1 score
- Output feature importance rankings and save the trained model

Expected Output:

```
=== Data Ingestion ===
Loaded 12,453 records (37 features)
Missing values filled: billing_amount (2.1%), last_login
(5.4%)
Engineered 14 new features (tenure_bin, avg_monthly_spend,
support_freq_ratio...)
```

```
=== Model Comparison (5-Fold Cross-Validation) ===
+-------------------------+-----------+-----------+-------
---+--------+
| Model                   | Accuracy  | Precision | Recall
| F1     |
+-------------------------+-----------+-----------+-------
---+--------+
| Logistic Regression     | 0.812     | 0.743     | 0.681
| 0.711  |
| Random Forest           | 0.874     | 0.831     | 0.789
| 0.809  |
| XGBoost (tuned)         | 0.891     | 0.856     | 0.812
| 0.833  |
| SVM (RBF kernel)        | 0.853     | 0.802     | 0.756
| 0.778  |
+-------------------------+-----------+-----------+-------
---+--------+

=== Best Model: XGBoost ===
Hyperparameters: {max_depth: 6, learning_rate: 0.05, n_est
imators: 350}

Top 5 Feature Importances:
  1. months_since_last_activity  — 0.187
  2. support_ticket_count        — 0.143
  3. avg_monthly_spend           — 0.121
  4. contract_type_monthly       — 0.098
  5. tenure_months               — 0.087

Model saved to models/churn_xgb_v2.pkl
```

---

6. Async API Gateway with Rate Limiting & Caching

Description: Build a reverse-proxy API gateway that routes requests to downstream microservices. Implement token-bucket rate limiting, response caching with TTL, and circuit-breaker patterns.

Prerequisites:

- `asyncio` event loop and coroutines
- `aiohttp` or `FastAPI` framework
- Token-bucket rate limiting algorithm
- Redis for caching (`aioredis`)
- Circuit breaker design pattern
- HTTP reverse proxy concepts
- Middleware pattern

Use-Case:

- Single entry point routing `/api/users/**`, `/api/orders/**`, etc. to separate services
- Enforce per-API-key rate limits (e.g., 50 req/min)
- Cache GET responses with configurable TTL
- Open circuit breaker after 5 consecutive failures on a downstream service
- Return fallback 503 responses when circuit is open
- Expose a health dashboard showing service status and cache hit rates

Expected Output:

```
=== Gateway Startup ===
[INFO] API Gateway running on http://0.0.0.0:8080
[INFO] Routes loaded:
       /api/users/**    -> http://user-service:3001
       /api/orders/**   -> http://order-service:3002
       /api/products/** -> http://product-service:3003

=== Request Log ===
```

```
[REQ] GET /api/products/42  client=api_key_9x3f
      -> CACHE HIT (TTL: 45s remaining) — 200 OK in 2ms

[REQ] GET /api/orders/latest  client=api_key_9x3f
      -> PROXY to order-service — 200 OK in 134ms

[REQ] POST /api/users/signup  client=api_key_b2k7
      -> RATE LIMITED (52/50 req/min) — 429 Too Many Reque
sts

[REQ] GET /api/orders/7891  client=api_key_m4n1
      -> CIRCUIT OPEN (order-service) — 503 Service Unavai
lable
        Fallback: {"error": "Service temporarily unavailab
le", "retry_after": 30}

=== Health Dashboard ===
+------------------+--------+---------+----------+--------
-----+
| Service          | Status | Latency | Circuit  | Cache H
its  |
+------------------+--------+---------+----------+--------
-----+
| user-service     | UP     | 89ms    | CLOSED   | 1,204
|
| order-service    | DOWN   | timeout | OPEN     | 302
|
| product-service  | UP     | 45ms    | CLOSED   | 8,912
|
+------------------+--------+---------+----------+--------
-----+
```

7. Compiler/Interpreter for a Mini Language

Description: Create a tokenizer (lexer), parser, AST builder, and tree-walking interpreter for a small language supporting variables, arithmetic, conditionals, loops, and functions.

Prerequisites:

- Formal grammars and BNF notation
- Recursive descent parsing
- Abstract Syntax Trees (AST) and the visitor pattern
- Tokenization / lexical analysis
- Scope and environment chains for variable resolution
- Recursion and tree traversal algorithms

Use-Case:

- Define a mini-language syntax (variables, `if/else`, `while`, `fn`)
- Lexer tokenizes source code into a stream of tokens
- Parser builds an AST from the token stream
- Interpreter walks the AST and executes instructions
- Support recursive functions (e.g., `fibonacci(10)`)
- Embed as a DSL inside a larger Python app for business rule evaluation

Expected Output:

```
=== Source Code (MiniLang) ===
fn fibonacci(n) {
    if n <= 1 { return n }
    return fibonacci(n - 1) + fibonacci(n - 2)
}
let result = fibonacci(10)
print("Fibonacci(10) = " + str(result))

=== Lexer Output ===
[FN, IDENT("fibonacci"), LPAREN, IDENT("n"), RPAREN, LBRAC
E,
```

```
 IF, IDENT("n"), LTE, INT(1), LBRACE, RETURN, IDENT("n"),
RBRACE,
 RETURN, IDENT("fibonacci"), LPAREN, IDENT("n"), MINUS, IN
T(1), RPAREN,
 PLUS, IDENT("fibonacci"), LPAREN, IDENT("n"), MINUS, INT
(2), RPAREN,
 RBRACE, LET, IDENT("result"), ASSIGN, IDENT("fibonacci"),
LPAREN,
 INT(10), RPAREN, PRINT, STRING("Fibonacci(10) = "), PLUS,
 IDENT("str"), LPAREN, IDENT("result"), RPAREN, EOF]

=== AST (abbreviated) ===
Program
├── FunctionDecl("fibonacci", params=["n"])
│   ├── IfStatement(condition=BinOp(<=, Ident("n"), Litera
l(1)))
│   │   └── ReturnStmt(Ident("n"))
│   └── ReturnStmt(BinOp(+, Call("fibonacci", ...), Call
("fibonacci", ...)))
└── LetDecl("result", Call("fibonacci", [Literal(10)]))
└── PrintStmt(BinOp(+, ...))

=== Interpreter Output ===
Fibonacci(10) = 55
```

---

8. Real-Time Data Streaming Dashboard

Description: Consume a live data stream, process it with windowed aggregations (moving average, anomaly detection), and push updates to a browser dashboard via SSE or WebSockets.

Prerequisites:

- Async generators and `asyncio`
- `FastAPI` or `Flask` with SSE/WebSocket support

- `pandas` for windowed aggregations
- Basic statistics (moving average, standard deviation, z-score)
- HTML/CSS/JavaScript for frontend charts (e.g., Chart.js)
- Event-driven architecture

Use-Case:

- Ingest sensor data (temperature, vibration) from a simulated IoT feed
- Compute 5-minute moving averages and z-scores per sensor
- Push updates to the browser every second via WebSocket
- Trigger alerts when a reading exceeds a configurable threshold
- Display live-updating line charts and an alert log in the browser

Expected Output:

```
=== Server Console ===
[INFO] Stream processor started — consuming from sensors/f
actory-a
[INFO] Dashboard available at http://localhost:5000/dashbo
ard
[INFO] 3 clients connected

=== Live Sensor Feed (every 1s) ===
[14:05:31] sensor-T1  temp=72.3F  vibration=0.12g  status=
NORMAL
[14:05:32] sensor-T1  temp=73.1F  vibration=0.14g  status=
NORMAL
[14:05:33] sensor-T1  temp=89.7F  vibration=0.31g  status=
WARNING
[14:05:34] sensor-T1  temp=104.2F vibration=0.58g  status=
CRITICAL

=== Alert Triggered ===
[ALERT] sensor-T1 — Temperature exceeded threshold (>100F)
        Current: 104.2F | 5-min avg: 82.4F | Deviation: +
```

```
2.7 sigma
        Action: Notification sent to ops-team@factory.com

=== Dashboard Widget (Browser) ===
┌──────────────────────────────────────────┐
│ Sensor T1 — Temperature (last 5 min)   │
│ 105│              /                     │
│  95│            /                       │
│  85│         /                          │
│  75│──────/────── threshold: 100F       │
│  65│    /                               │
│    └──────────────────────────────────  │
│     14:05:00            14:05:34        │
│ Moving Avg: 82.4F   |  Alerts: 1       │
└──────────────────────────────────────────┘
```

---

9. Plugin Architecture with Dynamic Module Loading

Description: Design an application core that discovers, loads, and manages plugins at runtime. Support lifecycle hooks, dependency resolution, and sandboxed execution.

Prerequisites:

- `importlib` and `importlib.metadata`
- Abstract base classes (`abc.ABC`, `@abstractmethod`)
- Decorators and class registries
- `pyproject.toml` entry points
- Dependency graph resolution (topological sort)
- Error handling and graceful degradation

Use-Case:

- Application scans a `./plugins/` directory on startup

- Each plugin implements a `PluginBase` interface with `activate()` / `deactivate()`
- Resolve inter-plugin dependencies before activation
- Plugins register new CLI commands, themes, or output formats
- Core app remains unmodified when adding/removing plugins

Expected Output:

```
=== Application Startup ===
$ sitegen build --theme dark-mode

[CORE] Scanning plugin directory: ./plugins/
[CORE] Discovered 4 plugins:
       ├── markdown-parser v2.1.0 (built-in)
       ├── dark-mode-theme v1.3.2 (third-party)
       ├── rss-feed v1.0.0 (third-party, depends: markdown
-parser)
       └── image-optimizer v0.9.1 (third-party)

[CORE] Resolving dependencies...
       markdown-parser    (no dependencies)          OK
       dark-mode-theme    (no dependencies)          OK
       rss-feed           -> markdown-parser         OK (s
atisfied)
       image-optimizer    (no dependencies)          OK

[CORE] Activating plugins in order...
       [1/4] markdown-parser.activate()  — registered: .md
-> HTML converter
       [2/4] dark-mode-theme.activate()  — registered: the
me "dark-mode"
       [3/4] rss-feed.activate()         — registered: com
mand "generate-rss"
       [4/4] image-optimizer.activate()  — registered: pos
t-processor for .png/.jpg
```

```
[CORE] Building site...
       Processed 24 pages | Theme: dark-mode | RSS: feed.x
ml generated
       Images optimized: 18 files, saved 4.2 MB
[CORE] Build complete -> ./dist/ (0.87s)
```

---

10. Blockchain Prototype

Description: Implement a simplified blockchain with proof-of-work consensus, transaction signing, peer-to-peer block propagation, and a wallet system.

Prerequisites:

- `hashlib` (SHA-256) for block hashing
- Public-key cryptography (`ecdsa` or `cryptography` library)
- Socket programming for peer-to-peer networking
- Merkle tree data structure
- Linked list / chain data structure
- `threading` or `asyncio` for concurrent node operation

Use-Case:

- Launch multiple nodes that discover each other on a local network
- Create and digitally sign transactions between wallets
- Nodes mine blocks by finding a nonce that satisfies the difficulty target
- Mined blocks broadcast to all peers via gossip protocol
- Each node independently validates and appends the block
- Query any node for wallet balances

Expected Output:

```
=== Node Startup (3 nodes) ===
[NODE-1] Listening on port 5001 | Wallet: 0xa3f8...c1d2
```

```
[NODE-2] Listening on port 5002 | Wallet: 0xb7d4...e5f6
[NODE-3] Listening on port 5003 | Wallet: 0xc9e2...a7b8

=== Transaction ===
[NODE-1] Creating transaction:
         From:   0xa3f8...c1d2
         To:     0xb7d4...e5f6
         Amount: 2.5 coins
         Signature: 3045022100...  Valid

=== Mining ===
[NODE-2] Mining block #7 (2 transactions in mempool)...
         Difficulty: 4 (hash must start with "0000")
         Nonce: 0      -> hash: 8a3f1b...     MISS
         Nonce: 1      -> hash: c72de9...     MISS
         ...
         Nonce: 48,231 -> hash: 0000a8f3c1... FOUND!

[NODE-2] Block #7 mined in 3.42s
         Hash:        0000a8f3c1d2b7e4...
         Prev Hash:   0000f1e2d3c4b5a6...
         Merkle Root: 7b3e9f...
         Transactions: 2
         Miner Reward: 1.0 coin -> 0xb7d4...e5f6

=== Propagation ===
[NODE-2] Broadcasting block #7 to peers...
[NODE-1] Received block #7 — validating... Accepted (chain
height: 7)
[NODE-3] Received block #7 — validating... Accepted (chain
height: 7)

=== Wallet Balances ===
0xa3f8...c1d2:  7.5 coins
```

```
0xb7d4...e5f6: 13.5 coins (includes mining rewards)
0xc9e2...a7b8:  4.0 coins
```

---

11. Automated Testing Framework

Description: Build a miniature test framework from scratch with test discovery, fixtures, parameterized tests, assertion introspection, and parallel execution.

Prerequisites:

- Decorators (`@test`, `@fixture`, `@skip`)
- `inspect` module for function/module introspection
- Context managers for setup/teardown
- `multiprocessing.Pool` for parallel execution
- Exception handling and traceback formatting
- String formatting for readable assertion diffs

Use-Case:

- Discover test functions by naming convention (`test_*`) or decorator
- Run setup/teardown fixtures scoped to session, module, or function
- Parameterize a single test with multiple input sets
- Display assertion diffs showing expected vs. actual values
- Run tests in parallel across N worker processes
- Print a summary with pass/fail/skip counts and timing

Expected Output:

```
$ minitest run tests/ --parallel 4 --verbose

=== Test Discovery ===
Found 23 tests across 5 modules
Fixtures loaded: db_connection (session), temp_dir (functi
```

```
on), mock_api (function)

=== Execution (4 workers) ===
tests/test_auth.py
  PASS  test_login_valid_credentials
[0.02s]
  PASS  test_login_invalid_password
[0.01s]
  FAIL  test_login_expired_token
[0.03s]
        AssertionError: Expected status=401, got status=20
0
        |  assert response.status == 401
        |         |               |
        |         200             401
        at tests/test_auth.py:45

tests/test_cart.py
  PASS  test_add_item[product_id=1, qty=1]
[0.01s]
  PASS  test_add_item[product_id=2, qty=5]
[0.01s]
  PASS  test_add_item[product_id=99, qty=0]
[0.01s]
  SKIP  test_checkout_stripe (skipped: no API key)
[0.00s]

...

=== Summary ===
23 tests | 20 passed | 2 failed | 1 skipped
Total time: 0.48s (parallel across 4 workers)
Slowest: test_full_integration (0.21s)
```

12. Graph Database Engine

Description: Implement an in-memory graph database with typed nodes/edges, a query language for traversals and pattern matching, and disk persistence via write-ahead logging.

Prerequisites:

- Graph theory (nodes, edges, adjacency lists)
- BFS, DFS, Dijkstra's shortest path algorithms
- Tokenizer/parser for a simple query language
- Indexing (hash index on node properties)
- File I/O and write-ahead log (WAL) pattern
- `json` or `msgpack` for serialization

Use-Case:

- Create nodes with labels and properties (e.g., `Person`, `Company`)
- Create typed, directed edges with properties (e.g., `FRIENDS_WITH`)
- Query multi-hop traversals with filters (e.g., friends-of-friends at company X)
- Compute shortest path between two nodes
- Persist all mutations to a WAL; recover state on restart
- Show database stats (node/edge counts, indexes, WAL size)

Expected Output:

```
=== Graph DB Shell ===
graphdb> CREATE NODE (alice:Person {name: "Alice", age: 3
0, city: "Austin"})
Node created: Person#1

graphdb> CREATE NODE (bob:Person {name: "Bob", age: 28, ci
ty: "Dallas"})
```

```
Node created: Person#2

graphdb> CREATE NODE (acme:Company {name: "Acme Corp", ind
ustry: "Tech"})
Node created: Company#3

graphdb> CREATE EDGE (alice)-[:FRIENDS_WITH {since: 2021}]
->(bob)
Edge created: Person#1 —FRIENDS_WITH-> Person#2

graphdb> CREATE EDGE (bob)-[:WORKS_AT {role: "Engineer"}]-
>(acme)
Edge created: Person#2 —WORKS_AT-> Company#3

graphdb> MATCH (p:Person)-[:FRIENDS_WITH]->()-[:WORKS_AT]-
>(c:Company)
         WHERE c.name = "Acme Corp"
         RETURN p.name, c.name

+----------+-----------+
| p.name   | c.name    |
+----------+-----------+
| Alice    | Acme Corp |
+----------+-----------+
1 row returned (traversal: 3 nodes, 2 edges) in 0.4ms

graphdb> SHORTEST_PATH (alice)-[*1..4]->(acme)
Path: Alice —FRIENDS_WITH-> Bob —WORKS_AT-> Acme Corp
Length: 2 hops | Total weight: 2.0

graphdb> STATS
Nodes: 3 | Edges: 2 | Indexes: 2 (Person.name, Company.nam
e)
WAL: 5 entries | Disk snapshot: 2 min ago
```

13. PDF Report Generator with Templating

Description: Build a system that takes structured data, applies it to customizable templates, and outputs polished PDF reports with tables, charts, and conditional sections.

Prerequisites:

- `Jinja2` for HTML/template rendering
- `ReportLab` or `WeasyPrint` for PDF generation
- `matplotlib` or `Plotly` for chart generation
- Database querying (`sqlite3` or `SQLAlchemy`)
- File I/O and path management (`pathlib`)
- `smtplib` for email delivery

Use-Case:

- Pull monthly sales data from a database
- Render data into a Jinja2 HTML template with header, summary table, and charts
- Conditionally include warning sections (e.g., declining regions)
- Convert rendered HTML to a multi-page PDF
- Auto-email the PDF attachment to a distribution list

Expected Output:

```
=== Report Generation ===
$ python generate_report.py --month 2026-01 --template sal
es_monthly

[1/5] Connecting to database... OK
[2/5] Querying January 2026 sales data... OK (3,412 record
s)
[3/5] Rendering template "sales_monthly"...
```

```
      - Header: "Monthly Sales Report — January 2026"
      - Summary Table: revenue, units sold, avg order valu
e
      - Bar Chart: revenue by region (North, South, East,
West)
      - Line Chart: daily sales trend
      - Conditional Section: "West region declined 12% Mo
M" (included)
      - Footer: page numbers, generation timestamp
[4/5] Generating PDF... OK
[5/5] Sending email...
      To: exec-team@company.com, sales-leads@company.com
      Subject: "January 2026 Sales Report"
      Attachment: sales_report_2026-01.pdf (2.4 MB, 6 page
s)
      Sent successfully

Output: reports/sales_report_2026-01.pdf

=== PDF Contents (page 1 preview) ===
┌──────────────────────────────────────────────────────┐
│        MONTHLY SALES REPORT                          │
│            January 2026                              │
├──────────────────────────────────────────────────────┤
│ Total Revenue:     $1,247,832                        │
│ Units Sold:        3,412                             │
│ Avg Order Value:   $365.72                           │
│ MoM Growth:        +8.3%                             │
│                                                      │
│  ┌──────────────────────────────┐                    │
│  │        Revenue by Region     │                    │
│  │ =====  North:  $412K         │                    │
│  │ ====   East:   $338K         │                    │
│  │ ===    South:  $309K         │                    │
│  │ ==     West:   $189K (!)     │                    │
│  └──────────────────────────────┘                    │
```

```
│                                        Page 1 of 6 │
└────────────────────────────────────────────────────┘
```

---

14. Concurrent Web Crawler with Depth Control

Description: Build a web crawler that starts from a seed URL, extracts links, and recursively crawls up to a configurable depth with concurrency, deduplication, and `robots.txt` compliance.

Prerequisites:

- `asyncio` and `aiohttp` for async HTTP requests
- BFS graph traversal algorithm
- `urllib.parse` for URL normalization and joining
- `robotparser` for `robots.txt` compliance
- Sets and queues for deduplication
- `json` for crawl graph export

Use-Case:

- Accept a seed URL, max depth, and concurrency limit as CLI args
- Crawl pages level by level using async BFS
- Skip already-visited URLs and disallowed paths
- Log status codes and flag broken links (404), redirects (301/302)
- Identify orphan pages with zero inbound links
- Export the full link graph as JSON and a sitemap as XML

Expected Output:

```
$ python crawler.py --seed https://example.com --depth 3 -
-concurrency 20

=== Crawl Started ===
```

```
[INFO] Seed: https://example.com
[INFO] Max depth: 3 | Concurrency: 20 | Respecting robots.
txt: YES

[DEPTH 0] https://example.com                          200
OK      0.12s
[DEPTH 1] https://example.com/about                    200
OK      0.09s
[DEPTH 1] https://example.com/products                 200
OK      0.15s
[DEPTH 1] https://example.com/blog                     200
OK      0.11s
[DEPTH 2] https://example.com/products/widget-pro      200
OK      0.08s
[DEPTH 2] https://example.com/blog/post-1              200
OK      0.10s
[DEPTH 2] https://example.com/old-promo                301
-> /products
[DEPTH 2] https://example.com/careers                  404
NOT FOUND
[DEPTH 3] https://example.com/blog/post-1/comments     200
OK      0.14s
...

=== Crawl Complete (12.4s) ===
Pages crawled:      147
Unique URLs found:  203
Skipped (robots):   12
Duplicates avoided: 56

=== SEO Audit Report ===
Broken Links (404):
  - /careers (linked from: /about, /footer)
  - /team/john (linked from: /about)

Redirect Chains (301/302):
```

```
  - /old-promo -> /products (1 hop)
  - /legacy/api -> /v1/api -> /v2/api (2 hops — consider f
ixing)

Orphan Pages (no inbound links):
  - /products/widget-legacy
  - /blog/draft-post-test

Crawl graph saved to: output/crawl_graph.json
Site map saved to:    output/sitemap.xml
```

---

15. Event-Driven Microservice with CQRS Pattern

Description: Implement a microservice separating read and write models (CQRS) with event sourcing. Commands emit domain events; handlers update read-optimized projections. Full event history is replayable.

Prerequisites:

- Domain-Driven Design (aggregates, commands, events)
- Event sourcing and append-only event stores
- Message bus / pub-sub (`Redis Streams`, `RabbitMQ`, or in-process)
- Eventual consistency concepts
- `asyncio` for async event handlers
- Two-database pattern (write store vs. read store)

Use-Case:

- `PlaceOrderCommand` creates an order aggregate and emits `OrderPlaced` event
- Event handlers asynchronously update a denormalized read model (dashboard-ready)
- Separate handlers send notification emails and update analytics

- Query side reads from the denormalized store with sub-millisecond latency
- Full event log supports auditing and state reconstruction via replay

Expected Output:

```
=== Command Side (Write) ===
>>> cmd = PlaceOrderCommand(customer_id="C-42", items=[
...     {"sku": "WIDGET-01", "qty": 3, "price": 29.99},
...     {"sku": "GADGET-05", "qty": 1, "price": 149.99}
... ])
>>> bus.dispatch(cmd)

[WRITE] PlaceOrderCommand received
[WRITE] Aggregate Order#ORD-1087 created
[EVENT STORE] Appended events:
  1. OrderPlaced       {order_id: "ORD-1087", customer: "C
-42", total: $239.96}
  2. InventoryReserved {sku: "WIDGET-01", qty: 3}
  3. InventoryReserved {sku: "GADGET-05", qty: 1}
[BUS] Published 3 events to "orders" topic

=== Event Handlers (Async) ===
[HANDLER: OrderDashboardProjection] OrderPlaced -> updatin
g read model...
  Read DB: INSERT INTO order_summary (id, customer, total,
status, item_count)
           VALUES ('ORD-1087', 'C-42', 239.96, 'PLACED',
4)

[HANDLER: NotificationService] OrderPlaced -> sending conf
irmation email...
  Email sent to customer C-42 OK

[HANDLER: AnalyticsProjection] OrderPlaced -> updating dai
ly stats...
```

```
  Today's revenue: $12,847.32 (+$239.96)

=== Query Side (Read) ===
>>> query = GetOrderSummary(order_id="ORD-1087")
>>> result = read_store.execute(query)
{
  "order_id": "ORD-1087",
  "customer_id": "C-42",
  "status": "PLACED",
  "total": 239.96,
  "item_count": 4,
  "placed_at": "2026-02-24T14:32:01Z"
}
Response time: 1.2ms (denormalized read model)

=== Event Replay (Audit) ===
>>> events = event_store.get_events(aggregate_id="ORD-108
7")
[Event #1] OrderPlaced       @ 14:32:01  {total: 239.96, s
tatus: PLACED}
[Event #2] OrderUpdated      @ 14:45:22  {removed: "GADGET
-05", new_total: 89.97}
[Event #3] PaymentProcessed  @ 14:46:01  {amount: 89.97, m
ethod: "card_ending_4242"}
[Event #4] OrderShipped      @ 15:10:33  {tracking: "1Z999
AA10123456784"}

>>> rebuild = event_store.replay("ORD-1087")
Reconstructed state: Order(id=ORD-1087, status=SHIPPED, to
tal=89.97, items=3)
```

▼ Python Resources

Python for Beginners (Full Course) | Programming Tutorial

Python Tutorial, Easy Python tutorial for beginner, learn Python Programming, learn python programming with example and syntax. Download python, install pyth...

https://youtube.com/playlist?list=PLsyeobzWxl7poL9JTVyndKe62ieoN-MZ3&si=TGvgkv6irQvqmWCp